

Thesis-1.pdf

 Indian Institute of Technology Jammu

Document Details

Submission ID

trn:oid::3117:586989914

Submission Date

May 6, 2026, 3:13 PM GMT+5:30

Download Date

May 6, 2026, 3:32 PM GMT+5:30

File Name

Thesis-1.pdf

File Size

2.9 MB

70 Pages

12,680 Words

66,532 Characters

43% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



71 AI-generated only 43%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



Auditor-Free Byzantine-Resilient Federated Learning via Functional Inner Products

DEEPAK GUPTA
2024PCS0024

A Dissertation Submitted to
Indian Institute of Technology, Jammu
In Partial Fulfillment of the Requirements for the Degree of
Master of Technology
in
Computer Science and Engineering



भारतीय प्रौद्योगिकी
संस्थान जम्मू
INDIAN INSTITUTE OF
TECHNOLOGY JAMMU

Department of Computer Science & Engineering
May 2026

DECLARATION

This thesis is submitted as a requirement for the award of the Master of Technology degree in Computer Science and Engineering in the Department of Computer Science and Engineering. I declare that this written submission represents my ideas in my own words, and where others' ideas or words have been included, I have adequately cited and referenced the original sources. The research embodied in this thesis has not been submitted in part or full, to any other University or Institute for the award of any other degree or diploma before. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the Institute and can also evoke penal action from the sources that have thus not been properly cited, or from whom proper permission has not been taken when needed.

Name: Deepak Gupta
Roll No.: 2024PCS0024

APPROVAL SHEET

This thesis titled **Auditor-Free Byzantine-Resilient Federated Learning via Functional Inner Products** by **Deepak Gupta** is approved for the degree of **Master of Technology in Computer Science and Engineering** from IIT Jammu.

Date:

Signature:_____

Name, Institute
External Examiner

Signature:_____

Name, IIT Jammu
Internal Examiner

Signature:_____

Name, IIT Jammu
Chairman

Signature:_____

Dr. Yamuna Prasad, IIT Jammu
Supervisor

*To my parents,
who gave me the courage to dream big,
and to my teachers,
who showed me how to turn those dreams into reality.*

Acknowledgement

I would like to take this opportunity to express my sincere gratitude to everyone whose support and guidance have made this dissertation possible. During this journey, each experiment, protocol refinement, and discussion helped me understand the research problem more clearly.

I am sincerely grateful to my supervisor, **Dr. Yamuna Prasad**, Head of the Department, Department of Computer Science and Engineering, IIT Jammu. His guidance helped me bring together privacy-preserving federated learning, applied cryptography, and adversarial machine-learning security into one focused thesis. He gave me the freedom to explore ideas, while also encouraging me to justify each assumption and improve the rigour of the work. I have learned a great deal from his patience, careful feedback, and high academic standards.

I am thankful to my fellow M.Tech and Ph.D. scholars at IIT Jammu for the late-evening conversations, debugging sessions, and research discussions that helped this work take shape. Those exchanges made the long process of experiments, writing, and revision more collaborative and memorable.

I thank the Department of Computer Science and Engineering at IIT Jammu for the academic environment, coursework, and compute support that made this study possible, and I thank the faculty members who taught and guided me during my first year.

Finally, I thank my parents and my family for their constant support and encouragement throughout this thesis.

Deepak Gupta

List of Publications

Conference Publications

1. Mahendra Kumar Gurve, Deepak Gupta, Nitin, and Yamuna Prasad, “Eliminating the Auditor: Functional Inner Product Based Byzantine-Resilient Federated Learning,” accepted as a poster at the *ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys 2026)*, Saint-Malo, France, May 2026.

Abstract

Federated Learning (FL) lets many clients train a shared model without exposing their raw data. To strengthen the privacy guarantee, modern deployments wrap each client's update in additive homomorphic encryption, giving rise to Privacy-Preserving Federated Learning (PPFL). The same encryption that hides gradients from a curious server, however, also hides whether they are honest: a Byzantine client can flip labels before training and quietly poison the aggregated model, either degrading overall accuracy or installing a backdoor on a chosen target class.

Existing defences in this setting rely on auditor-based architectures that insert a trusted third party to decrypt cluster-level statistics of the incoming updates and screen Byzantine clients with distributional models. While effective, such schemes reintroduce the very trust anchor PPFL was designed to remove, expose coarse structural information about the encrypted gradients, and scale super-linearly in the model dimension. A defence that is simultaneously fully encrypted, free of any new trust anchor, and linear in the model dimension has remained an open challenge.

This thesis develops **FIP-FL**, an auditor-free defence for the encrypted-gradient setting. Instead of clustering decrypted summaries, the verifier computes, under Paillier encryption, one projection score for each client: the *Functional Inner Product* (FIP) between that client's update and a momentum-tracked reference direction. The decrypted scalar then passes through the two checks used in my implementation and poster flow: first a direction filter, then a median-MAD magnitude window. I analyse the resulting protocol in terms of scalar-gradient privacy, the admissible region left to an attacker, and descent under an L -smooth objective.

The framework is implemented end-to-end and evaluated on MNIST and KDDCup99, under both IID and non-IID client partitions, and against targeted and untargeted label-flipping attacks at malicious-client rates up to 50%. Across this evaluation grid, FIP-FL achieves strong robustness without any external auditor. The research underlying this thesis has been accepted as a poster at the ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys 2026), Saint-Malo, France, May 2026.

Contents

List of Publications	xi
Abstract	xiii
List of Figures	xvii
List of Tables	xix
List of Abbreviations	xxi
1 Introduction	1
1.1 Background and Context	1
1.2 The Research Gap	1
1.3 Research Objectives and Contributions	2
2 Background and Motivation	3
2.1 Federated Learning and FedAvg	3
2.2 IID and Non-IID Federations	3
2.3 Privacy-Preserving Federated Learning	4
2.4 Poisoning Attack Taxonomy	4
3 Literature Survey	5
3.1 Robust Aggregation Rules	5
3.2 Detection-Based Defences	5
3.3 Auditor-Based PPFL Baseline	6
3.4 Positioning of FIP-FL	6
4 Preliminaries	7
4.1 Notation	7
4.2 The Paillier Cryptosystem	7
4.3 Functional Inner Product	8
4.4 Robust Statistics	9
4.5 System and Threat Model	9
4.6 Security and Privacy Goals	9
5 Proposed Framework: FIP-FL	11
5.1 System Architecture	11
5.2 System Setup and Key Distribution	13

5.3	Per-Round Protocol	15
5.3.1	Phase I: Local Training and Encrypted Update Generation	16
5.3.2	Phase II: Homomorphic FIP Score Computation	18
5.3.3	Phase III: Two-Stage Geometric Audit	20
5.3.4	Phase IV: Secure Aggregation	22
5.3.5	Phase V: Reference Update and Model Distribution	23
5.4	Theoretical Analysis	25
5.4.1	Gradient Privacy	26
5.4.2	Attack Resilience	26
5.4.3	Convergence Guarantee	26
6	Experimental Setup	29
6.1	Datasets	29
6.2	Data Partitioning	30
6.3	Attack Models	30
6.4	Evaluation Metrics	31
6.5	Implementation and Hardware	32
7	Results and Analysis	33
7.1	Non-IID Partition, Untargeted Attacks	33
7.1.1	MNIST	33
7.1.2	KDDCup99	34
7.2	Non-IID Partition, Targeted Attacks	34
7.2.1	MNIST	34
7.2.2	KDDCup99	35
7.3	IID Partition, Untargeted Attacks	36
7.3.1	MNIST	36
7.3.2	KDDCup99	37
7.4	IID Partition, Targeted Attacks	38
7.4.1	MNIST	38
7.4.2	KDDCup99	38
7.5	Aggregate Findings Across the Grid	39
8	Comparative Analysis	41
8.1	Privacy	41
8.2	Computational Complexity	42
8.3	Communication Complexity	42
8.4	Robustness and Engineering Trade-Offs	43
8.5	Scheme-versus-Scheme Accuracy Comparison	43
9	Conclusion and Future Work	45
9.1	Summary of the Thesis	45
9.2	Limitations and Future Work	45

List of Figures

5.1 Geometric architecture of the FIP-FL protocol. 12

7.1 Non-IID untargeted attack dynamics over 300 communication rounds. Accuracy converges stably for both datasets, while the direction/magnitude audit reaches perfect malicious-client detection by the final round. 35

7.2 KDDCup99 non-IID targeted attack results. The target-class trajectory remains high through training, and the round-300 comparison shows that overall and target-class accuracy stay stable even as the malicious-client fraction increases. 36

7.3 IID untargeted attack dynamics over 300 communication rounds. Both datasets show stable convergence, and the audit maintains perfect malicious-client detection without false positives at the reported final round. 37

7.4 IID targeted attack target-class accuracy over 300 communication rounds. In both datasets, the target class recovers to a stable high-accuracy regime by the final round. 39

List of Tables

6.1	Datasets used in the evaluation of FIP-FL.	29
7.1	MNIST, non-IID partition, untargeted label-flipping attack. . .	33
7.2	KDDCup99, non-IID partition (Dirichlet $\alpha = 0.5$), untargeted label-flipping attack.	34
7.3	MNIST, non-IID, targeted label-flipping attack ($c_s = 0, c_t = 7$); SenSys 2026 headline targeted-accuracy cell.	34
7.4	KDDCup99, non-IID partition (2 shards/client), targeted label-flipping attack.	35
7.5	MNIST, IID partition, untargeted label-flipping attack.	36
7.6	KDDCup99, IID partition, untargeted label-flipping attack. . . .	37
7.7	MNIST, IID partition, targeted label-flipping attack ($c_s = 0, c_t = 7$).	38
7.8	KDDCup99, IID partition, targeted label-flipping attack.	38
8.1	Per-round computational cost. N : clients; $ \mathcal{A} $: accepted clients; d : model dimension; B : batch size; E : local epochs.	42
8.2	Per-round communication cost by directed link.	42
8.3	Accuracy on MNIST non-IID under a 50% malicious-client rate. Entries are percentages; baseline values are from [1].	43

List of Abbreviations

ACC	— Overall Accuracy
BFV	— Brakerski–Fan–Vercauteren (homomorphic encryption scheme)
BGV	— Brakerski–Gentry–Vaikuntanathan (homomorphic encryption scheme)
CKKS	— Cheon–Kim–Kim–Song (approximate homomorphic encryption)
DCR	— Decisional Composite Residuosity
DoS	— Denial of Service
DP	— Differential Privacy
FedAvg	— Federated Averaging
FHE	— Fully Homomorphic Encryption
FIP	— Functional Inner Product
FIP-FL	— Functional Inner Product Federated Learning (proposed framework)
FL	— Federated Learning
FPR	— False Positive Rate
GMM	— Gaussian Mixture Model
HE	— Homomorphic Encryption
IID	— Independent and Identically Distributed
KDD	— Knowledge Discovery in Databases
MAD	— Median Absolute Deviation
MNIST	— Modified National Institute of Standards and Technology (database)
O_{acc}	— Other-label Accuracy
PCA	— Principal Component Analysis
PPFL	— Privacy-Preserving Federated Learning
R2L	— Remote-to-Local (KDDCup attack class)
SenSys	— ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems
SGD	— Stochastic Gradient Descent
SHA	— Secure Hash Algorithm
SMPC	— Secure Multiparty Computation
T_{acc}	— Target-class Accuracy
TF-IDF	— Term Frequency – Inverse Document Frequency
TKA	— Trusted Key Authority
TPR	— True Positive Rate
U2R	— User-to-Root (KDDCup attack class)

1 Introduction

1.1 Background and Context

Federated Learning (FL), introduced by McMahan et al. [2], is the training model I use when data cannot be pooled at one server. In my experiments, each client keeps its MNIST or KDDCup99 shard locally, computes an update from that shard, and sends only the update for global aggregation.

That update still carries information about the local data. Gradient reconstruction [3], membership inference, and property inference attacks show why simply keeping raw samples on the client is not enough. Privacy-Preserving Federated Learning (PPFL) therefore protects the update channel. I focus on additive homomorphic encryption, where the server can add encrypted updates while the individual gradient coordinates remain hidden.

1.2 The Research Gap

The difficulty is that encryption hides both useful updates and poisoned ones. If a malicious client trains on flipped labels or scales its update, the server sees a valid ciphertext vector and little else. Plaintext defences such as Krum [8], coordinate-wise median/trimmed-mean [9], and Bulyan [10] need distances or coordinate ordering. Detection methods such as Auror [11], FL-Trust [12], and PEFL [13] also rely on plaintext updates or summaries.

The closest encrypted-gradient baseline is Yazdinejad et al. [1]. Their scheme is strong, but it adds a trusted auditor, decrypts cluster summaries, uses an $\mathcal{O}(d^3)$ covariance inversion, and can expose non-IID population structure. My question is narrower: can we filter Byzantine clients while each individual update

stays encrypted, no auditor is added, and verifier work remains near-linear in d ?

1.3 Research Objectives and Contributions

This thesis designs and evaluates FIP-FL for that setting. The verifier computes a functional inner product (FIP) between each encrypted update and a trusted reference direction. Only the scalar score is decrypted. The audit then asks two questions: is the update pointing in the trusted direction, and is its projected magnitude within the round's robust window?

The main contributions are:

1. **Auditor-free encrypted verification.** FIP-FL replaces decrypted cluster statistics with one homomorphic projection score per client.
2. **Two-stage geometric audit.** The first check removes direction reversals; the second uses median-MAD statistics to remove weak or over-scaled positive scores.
3. **Formal analysis.** I analyse the scalar information revealed by FIP, the region an attacker must satisfy to pass the filter, the descent condition under L -smoothness, and the verifier complexity.
4. **Empirical validation.** The experiments use MNIST and KDDCup99 under IID/non-IID partitions, targeted/untargeted label flipping, and malicious-client rates up to 50%.
5. **Engineering efficiency.** The verifier avoids the $\mathcal{O}(d^3)$ covariance step and runs Phase II in parallel, giving a $13\times$ speed-up on the 25-core workstation.

The research underlying this thesis has been accepted as a poster at the ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys 2026), Saint-Malo, France, May 2026.

2 Background and Motivation

2.1 Federated Learning and FedAvg

Consider N clients, where client i holds private data $\mathcal{D}_i = \{(x_k^{(i)}, y_k^{(i)})\}_{k=1}^{n_i}$ and $n = \sum_i n_i$. The federated objective is the weighted empirical risk

$$F(\mathbf{w}) = \sum_{i=1}^N \frac{n_i}{n} F_i(\mathbf{w}), \quad F_i(\mathbf{w}) = \frac{1}{n_i} \sum_{k=1}^{n_i} \ell(f_{\mathbf{w}}(x_k^{(i)}), y_k^{(i)}). \quad (2.1)$$

FedAvg [2] optimises this objective by broadcasting $\mathbf{w}^{(t)}$, running local SGD at each client, and aggregating returned updates. With descent-direction update $\Delta\theta_i^{(t)} = \mathbf{w}^{(t)} - \mathbf{w}_i^{(t+1)}$, the server update is

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \sum_{i=1}^N \frac{n_i}{n} \Delta\theta_i^{(t)}. \quad (2.2)$$

The server therefore needs only coordinate-wise aggregation, while raw samples remain local.

2.2 IID and Non-IID Federations

FedAvg is simplest when client data are IID draws from a common population. Real federations are usually non-IID: users, hospitals, and network sites observe different label distributions and feature patterns. This thesis uses label skew, where each client receives a subset of labels with controlled overlap. Label skew makes local objectives F_i drift from the global objective F , so honest gradients can differ substantially. A Byzantine defence must therefore reject malicious deviation without treating legitimate heterogeneity as an attack.

2.3 Privacy-Preserving Federated Learning

FL does not automatically protect the update vector. Reconstruction, membership inference, and property inference attacks can all use gradients as side channels [3]. PPFL methods protect that channel in different ways: differential privacy clips and noises the update, secure aggregation [14] hides individuals but usually reveals only the sum, and homomorphic encryption allows limited computation directly over ciphertexts. Paillier [4] supports encrypted addition and multiplication by known scalars; BFV/CKKS [5] support richer packed arithmetic but are heavier than what this thesis needs.

I choose Paillier because FIP-FL's verifier performs a linear projection. The encrypted client update is multiplied by plaintext reference coordinates and accumulated into one encrypted score. This avoids DP noise and avoids a peer-to-peer secure-aggregation protocol while keeping the audit compatible with encrypted updates.

2.4 Poisoning Attack Taxonomy

A Byzantine client can submit a ciphertext vector that looks well formed, so syntax checks are not enough. I evaluate two pre-training label-flipping attacks. In the *untargeted* case, the attacker remaps every label by a fixed cyclic offset and tries to reduce overall accuracy. In the *targeted* case, it changes one source class to one target class; the global accuracy may still look acceptable, while the chosen class is damaged. Sign flipping, scaling, model replacement, and collusion are not separate experiment rows, but they explain why the audit needs both a direction check and a magnitude window.

3 Literature Survey

3.1 Robust Aggregation Rules

Robust aggregation replaces the FedAvg mean with a more defensive statistic. Krum and Multi-Krum [8] select updates that are near many other updates, so they require plaintext pairwise distances and cost $\mathcal{O}(N^2d)$. Coordinate-wise median and trimmed-mean [9] lower the cost to $\mathcal{O}(Nd \log N)$, but under label skew the honest coordinates need not form a single tight distribution. Bulyan [10] improves robustness by combining Krum-style selection with trimming, at the cost of the same plaintext access and client-count assumptions.

For an encrypted-update pipeline, these methods indicate what the filter should achieve but not how to implement it. Sorting coordinates, measuring distances, or selecting nearest neighbours is not available unless another party decrypts information.

3.2 Detection-Based Defences

Detection-based defences remove suspicious clients before aggregation. Auror [11] clusters update features and rejects minority clusters, which can be risky when non-IID honest clients also form multiple clusters. FL-Trust [12] scores plaintext updates against a trusted server gradient; this is conceptually close to FIP-FL, but it assumes the server can see the update. PEFL [13] considers encrypted correlation scoring, while Yazdinejad et al. [1] report weakness at high attack rates and non-IID settings. FoolsGold [15] uses historical similarity, and ShieldFL [16] relies on helper-assisted secure computation.

The pattern I take from this literature is practical: stronger screening usually

asks for plaintext access, another trust party, or more protocol communication. FIP-FL is aimed at the smaller design point where the update stays encrypted and only one scalar per client is revealed.

3.3 Auditor-Based PPFL Baseline

Yazdinejad et al. [1] are the closest baseline for my work because their pipeline also handles encrypted client updates. Their system places a trusted auditor between the clients and the aggregation server. The auditor decrypts cluster-level summaries, fits a Gaussian mixture model in a projected space, and uses Mahalanobis distance before the selected updates move to homomorphic aggregation.

I use their MNIST non-IID targeted result, about 96.5% target-class accuracy at a 50% attack rate, as the main comparison point. The reason to move beyond this baseline is not that it fails experimentally; rather, it changes the trust model, performs an $\mathcal{O}(d^3)$ covariance inversion, and decrypts statistics that can describe the non-IID client population.

3.4 Positioning of FIP-FL

FIP-FL makes four choices relative to this literature. Individual updates remain encrypted; no auditor/helper is added; the test is geometric alignment rather than clustering; and the verifier's main work is linear in d , with only scalar-score statistics after that. Thus the method keeps the privacy goal of PPFL, removes the auditor's decrypted cluster view, and still keeps a root-gradient-style alignment signal through homomorphic computation. The next chapter fixes the notation and primitives used by this design.

4 Preliminaries

4.1 Notation

Scalars are lowercase italics (a, η), vectors are lowercase bold ($\mathbf{x}, \boldsymbol{\theta}$), and matrices are uppercase bold ($\mathbf{W}, \mathbf{M}$). For $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$,

$$\langle \mathbf{a}, \mathbf{b} \rangle = \sum_{k=1}^d a_k b_k, \quad \|\mathbf{a}\|_2 = \sqrt{\langle \mathbf{a}, \mathbf{a} \rangle}. \quad (4.1)$$

A Paillier ciphertext is written $\llbracket m \rrbracket = \text{Enc}_{pk}(m)$. The main FL symbols are N clients, model dimension d , round index t , local epochs E , batch size B , learning rate η , and Paillier security parameter λ .

4.2 The Paillier Cryptosystem

Paillier [4] is the additive homomorphic scheme used in the implementation. For security parameter λ , the Trusted Key Authority samples primes p, q , sets $n = pq$, uses generator $g = n + 1$, and keeps the secret key $sk = (\ell, \alpha)$ with $\ell = \text{lcm}(p-1, q-1)$ and $\alpha = \ell^{-1} \bmod n$. In all experiments, $\lambda = 1024$.

For plaintext $m \in \mathbb{Z}_n$ and random $r \in \mathbb{Z}_n^*$,

$$\text{Enc}_{pk}(m) = g^m r^n \pmod{n^2}, \quad (4.2)$$

and decryption is

$$\text{Dec}_{sk}(c) = L(c^\ell \bmod n^2) \alpha \pmod{n}, \quad L(x) = \frac{x-1}{n}. \quad (4.3)$$

For plaintexts m_1, m_2 and known integer a ,

$$\text{Enc}_{pk}(m_1)\text{Enc}_{pk}(m_2) \equiv \text{Enc}_{pk}(m_1 + m_2) \pmod{n^2}, \quad (4.4)$$

$$\text{Enc}_{pk}(m_1)^a \equiv \text{Enc}_{pk}(am_1) \pmod{n^2}. \quad (4.5)$$

These two identities cover the operations used later in FIP-FL: encrypted coordinate sums and multiplication of an encrypted coordinate by a known reference-coordinate scalar. The protocol never requires multiplying two ciphertexts.

Real gradients are encoded as fixed-point integers. For scale s ,

$$\text{encode}(x) = \text{round}(sx), \quad \text{decode}(z) = z/s. \quad (4.6)$$

Signed values use centered modular encoding in the Paillier plaintext ring: $-a$ is stored as $n - a$ and decoded from $(-n/2, n/2]$. Negative plaintext scalars are handled through modular inverses in the exponentiation step. I use fixed-point scale $s = 10,000$. Since a FIP score contains one encoded gradient coordinate and one encoded reference coordinate, the decrypted scalar is divided by s^2 .

4.3 Functional Inner Product

For gradient vectors, the Functional Inner Product used in this thesis is the Euclidean dot product

$$\langle \mathbf{g}_i, \mathbf{g}_{\text{ref}} \rangle = \sum_{k=1}^d g_i^{(k)} g_{\text{ref}}^{(k)}. \quad (4.7)$$

It measures whether a client update aligns with the trusted reference direction. Combining (4.7) with Paillier scalar multiplication and addition gives the homomorphic score

$$\llbracket \langle \mathbf{g}_i, \mathbf{g}_{\text{ref}} \rangle \rrbracket = \prod_{k=1}^d \llbracket g_i^{(k)} \rrbracket^{\text{encode}(g_{\text{ref}}^{(k)})} \pmod{n^2}. \quad (4.8)$$

The TKA decrypts only this scalar. The server never observes a coordinate of $\mathbf{g}_i$.

4.4 Robust Statistics

The audit thresholds one scalar per client, so the robust statistics are one-dimensional. For score set $\mathcal{S} = \{s_1, \dots, s_N\}$,

$$\tilde{\mu} = \text{median}(\mathcal{S}), \quad \text{MAD} = \text{median}\{|s_i - \tilde{\mu}| : s_i \in \mathcal{S}\}. \quad (4.9)$$

I use the median and MAD because they are stable near the 50% contamination boundary. FIP-FL uses the adaptive window

$$\tau_{\min}^{(t)} = \alpha(\rho^{(t)})\tilde{\mu}^{(t)}, \quad \tau_{\max}^{(t)} = \tilde{\mu}^{(t)} + \gamma_{\text{mad}}\text{MAD}^{(t)}, \quad (4.10)$$

with $\gamma_{\text{mad}} = 5$. A score below the lower floor is too weak along the reference direction; a score above the upper cap is aligned but unusually influential.

4.5 System and Threat Model

FIP-FL uses four roles. Clients train locally and submit encrypted gradients. The Aggregation Server combines only accepted ciphertexts and broadcasts the model. The FIP Verifier computes scalar scores and applies the audit. The Trusted Key Authority generates Paillier keys and decrypts only protocol-specified scalar scores or aggregate updates. The Verifier can be co-located with the server, but it is not an external auditor because it never receives plaintext client gradients.

The adversary controls up to $f < N/2$ Byzantine clients. It may relabel local data, submit arbitrary valid ciphertexts, collude across malicious clients, and observe public broadcasts. The experiments instantiate this model with targeted and untargeted label flipping at malicious rates $f/N \in \{10\%, 20\%, 30\%, 50\%\}$.

4.6 Security and Privacy Goals

The protocol targets four goals. *Privacy*: the server and Verifier should not learn a client-gradient coordinate, norm, or pairwise distance beyond $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{g}_{\text{ref}}^{(t)} \rangle$. *Byzantine resilience*: accepted updates must have positive reference projection and score inside $(\tau_{\min}^{(t)}, \tau_{\max}^{(t)})$. *Convergence*: the aggregate should meet the descent

alignment and step-size condition in Chapter 5. *Efficiency*: verifier work should be $\mathcal{O}(Nd + N \log N)$ and aggregation $\mathcal{O}(|\mathcal{A}^{(t)}|d)$.

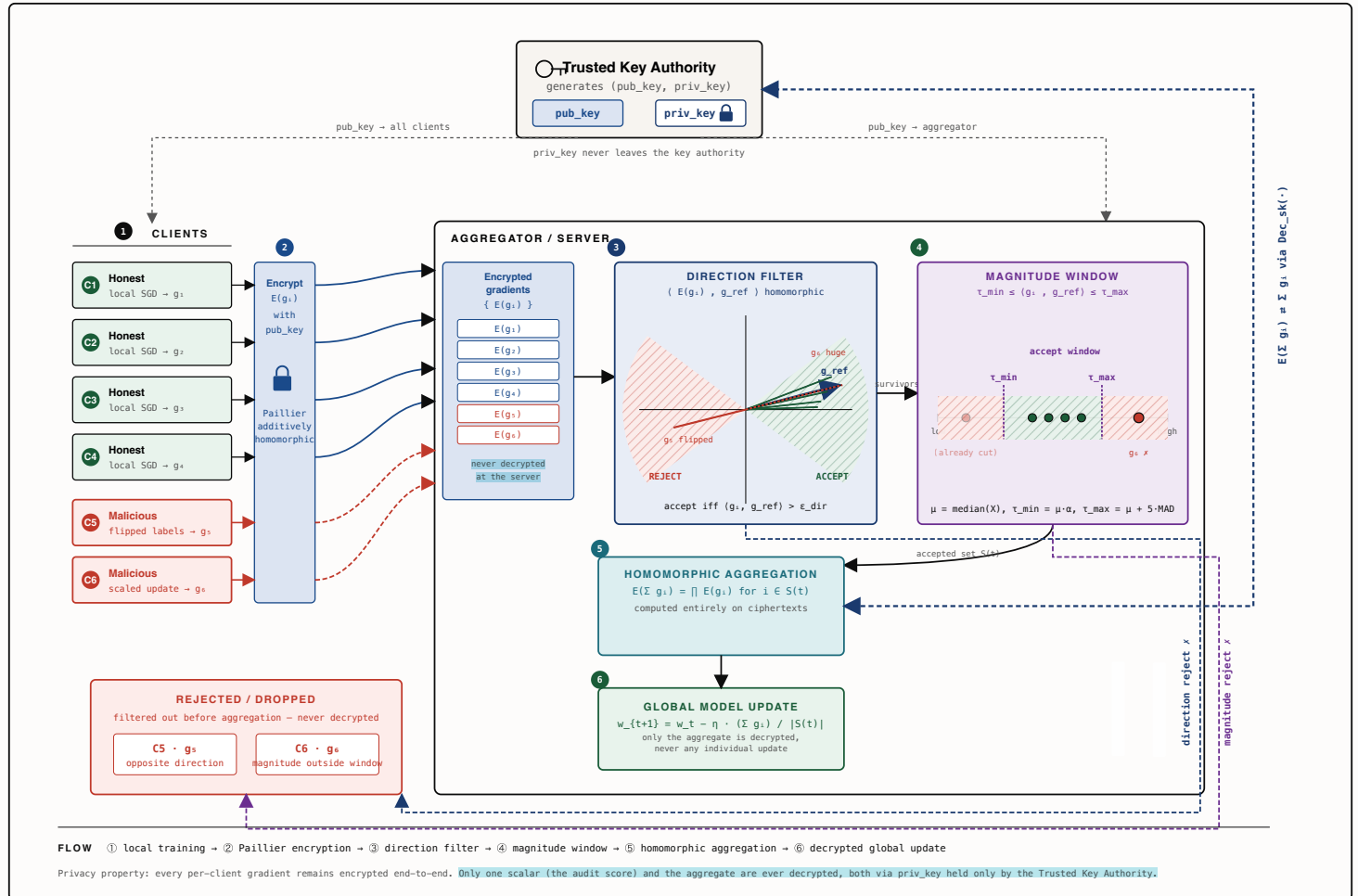
5 Proposed Framework: FIP-FL

5.1 System Architecture

FIP-FL has four logical entities: N clients, an Aggregation Server, a FIP Verifier, and a Trusted Key Authority (TKA). Figure 5.1 shows how I use them in one round: clients submit encrypted updates, the Verifier computes one FIP score per update, the audit keeps only acceptable ciphertexts, and the TKA decrypts scalar scores or the final aggregate only.

FIP-PPFL — Functional Inner Product · Privacy-Preserving Federated Learning

A trusted key authority publishes a Paillier public key. Clients train locally and submit encrypted gradients. The aggregator audits each encrypted update against a benign reference using two geometric criteria — first a directional check, then a magnitude window — and homomorphically aggregates only the survivors. The single private key never leaves the key authority.



① TRUSTED KEY AUTHORITY

Generates one Paillier keypair. Distributes the public key to all clients and the aggregator; keeps the private key sealed and uses it only for decryption of the audit score and the aggregate.

$pub_key \rightarrow$ broadcast
 $priv_key \rightarrow$ never leaves

③ DIRECTION FILTER

Server computes the homomorphic inner product $\langle g_i, g_ref \rangle$ against a benign reference built on public data. Updates pointing in the wrong half-space are removed before any aggregation.

accept iff $\langle g_i, g_ref \rangle > \epsilon_dir$

④ MAGNITUDE WINDOW

Robust statistics on the alignment scores. Median μ and MAD define an adaptive window; any client whose score is too small or too large is dropped — symmetric protection against shrinking and scaling attacks.

$\tau_min \leq \langle g_i, g_ref \rangle \leq \tau_max$

⑤ SECURE AGGREGATION

The encrypted gradients of the surviving set $S(t)$ are summed homomorphically. Only the aggregate is sent to the key authority for decryption — individual gradients never appear in plaintext.

$E(\sum g_i) = \prod E(g_i), i \in S(t)$

The roles are separated to avoid the auditor-style trust point. The Verifier sees ciphertexts and $\mathbf{R}^{(t)}$, but no plaintext client gradient. The TKA decrypts scalar scores and aggregate updates, but it does not choose which clients to reject.

5.2 System Setup and Key Distribution

Setup runs once before round 1. Algorithm 1 keeps three states separate: Paillier keys at the TKA, model parameters at the server, and the first unit-norm reference at the Verifier.

FIP-PPFL System Initialization and Reference Bootstrapping

A geometric view of the three setup phases — key generation & registration, model initialization, and reference bootstrapping — culminating in a unit-norm trusted reference direction $g_{\text{ref}}^{(o)} \in \mathbb{R}^d$ stored at the server-side FIP Verifier module for round 1.

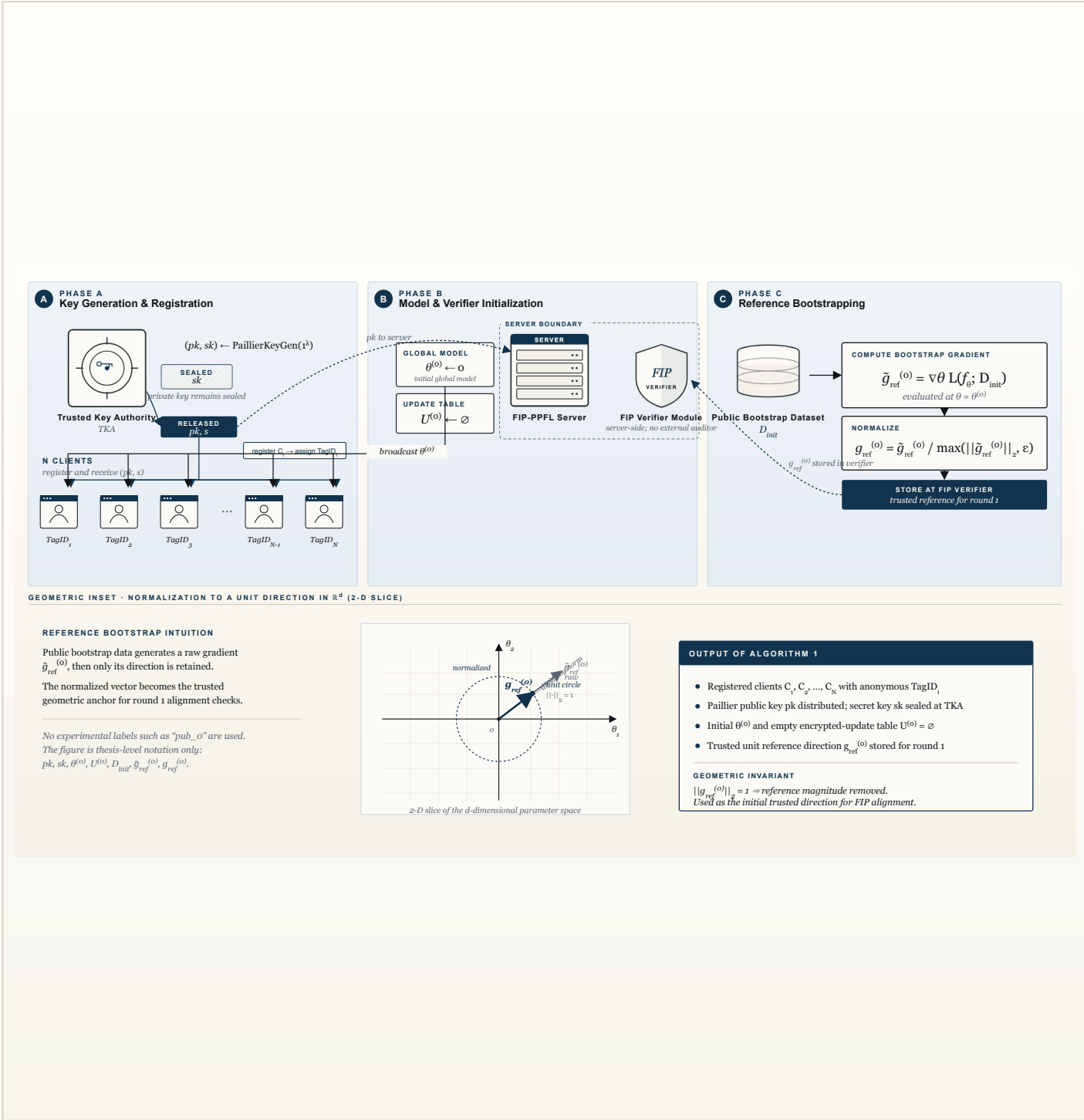


Figure 1. Algorithm 1. The TKA generates the Paillier key pair and keeps sk sealed while releasing only pk and the fixed-point scale S . The server initializes $\theta^{(o)} \leftarrow \text{InitModel}(f_\theta)$ (zero initialization for the logistic-regression experiments) together with $U^{(o)} = \emptyset$, broadcasts $\theta^{(o)}$ to the registered clients, and the server-side FIP Verifier stores the initial trusted reference direction. The trusted public bootstrap set D_{init} produces the raw gradient $\tilde{g}_{\text{ref}}^{(o)}$, which is normalized to $g_{\text{ref}}^{(o)}$ using $\max(\|\tilde{g}_{\text{ref}}^{(o)}\|_2, \epsilon)$. The bottom inset shows the geometric normalization from a raw gradient vector to a unit direction in a 2-D slice of the parameter

Algorithm 1 FIP-PPFL System Initialization and Reference Bootstrapping

Require: Number of clients N , model f_{θ} , public bootstrap set $\mathcal{D}_{\text{init}}$, security parameter λ , fixed-point scale s

Ensure: Registered clients, Paillier keys, initial global model $\theta^{(0)}$, trusted reference direction $\mathbf{g}_{\text{ref}}^{(0)}$

Key generation and registration

- 1: $(pk, sk) \leftarrow \text{PAILLIERKEYGEN}(1^\lambda)$ at the Trusted Key Authority
- 2: TKA keeps sk sealed and releases only pk to the clients and the FIP-PPFL server
- 3: **for** $i \leftarrow 1$ **to** N **do**
- 4: Register client C_i and assign anonymous tag TagID_i
- 5: Send (pk, s) to client C_i
- 6: **end for**

Model and verifier initialisation

- 7: Initialise global model parameters $\theta^{(0)} \leftarrow \mathbf{0}$
- 8: Broadcast $\theta^{(0)}$ to all registered clients
- 9: Initialise encrypted-update table $\mathcal{U}^{(0)} \leftarrow \emptyset$

Reference bootstrapping

- 10: Compute the trusted bootstrap gradient on public data:
- 11: $\tilde{\mathbf{g}}_{\text{ref}}^{(0)} \leftarrow \nabla_{\theta} \mathcal{L}(f_{\theta}; \mathcal{D}_{\text{init}}) \big|_{\theta=\theta^{(0)}}$
- 12: Normalise the reference direction:
- 13: $\mathbf{g}_{\text{ref}}^{(0)} \leftarrow \tilde{\mathbf{g}}_{\text{ref}}^{(0)} / \max\left(\left\|\tilde{\mathbf{g}}_{\text{ref}}^{(0)}\right\|_2, \varepsilon\right)$
- 14: Store $\mathbf{g}_{\text{ref}}^{(0)}$ at the FIP Verifier for round 1

The public bootstrap set is used only to obtain $\mathbf{g}_{\text{ref}}^{(0)}$ at the zero model. From round t onward, I write the same reference as $\mathbf{R}^{(t)}$.

5.3 Per-Round Protocol

Each round $t \geq 1$ follows the poster flow: encrypted gradient, FIP score, direction filter, magnitude window, secure aggregation, and reference update.

5.3.1 Phase I: Local Training and Encrypted Update Generation

Each client trains for E local epochs, averages mini-batch gradients, flattens the weight and bias terms into a d -dimensional vector, encrypts each coordinate, and sends the ciphertext vector to the Verifier. Algorithm 2 gives the honest path; malicious clients alter labels before this step.

Client-Side Local Training and Encrypted Gradient Submission

A geometric view of the client-side round procedure — receiving the round model, performing local mini-batch training, averaging and flattening the local gradient, encoding each coordinate with fixed-point scale S , encrypting under Paillier public key pk , and sending the anonymous encrypted submission $M_i^{(t)}$ to the FIP Verifier.

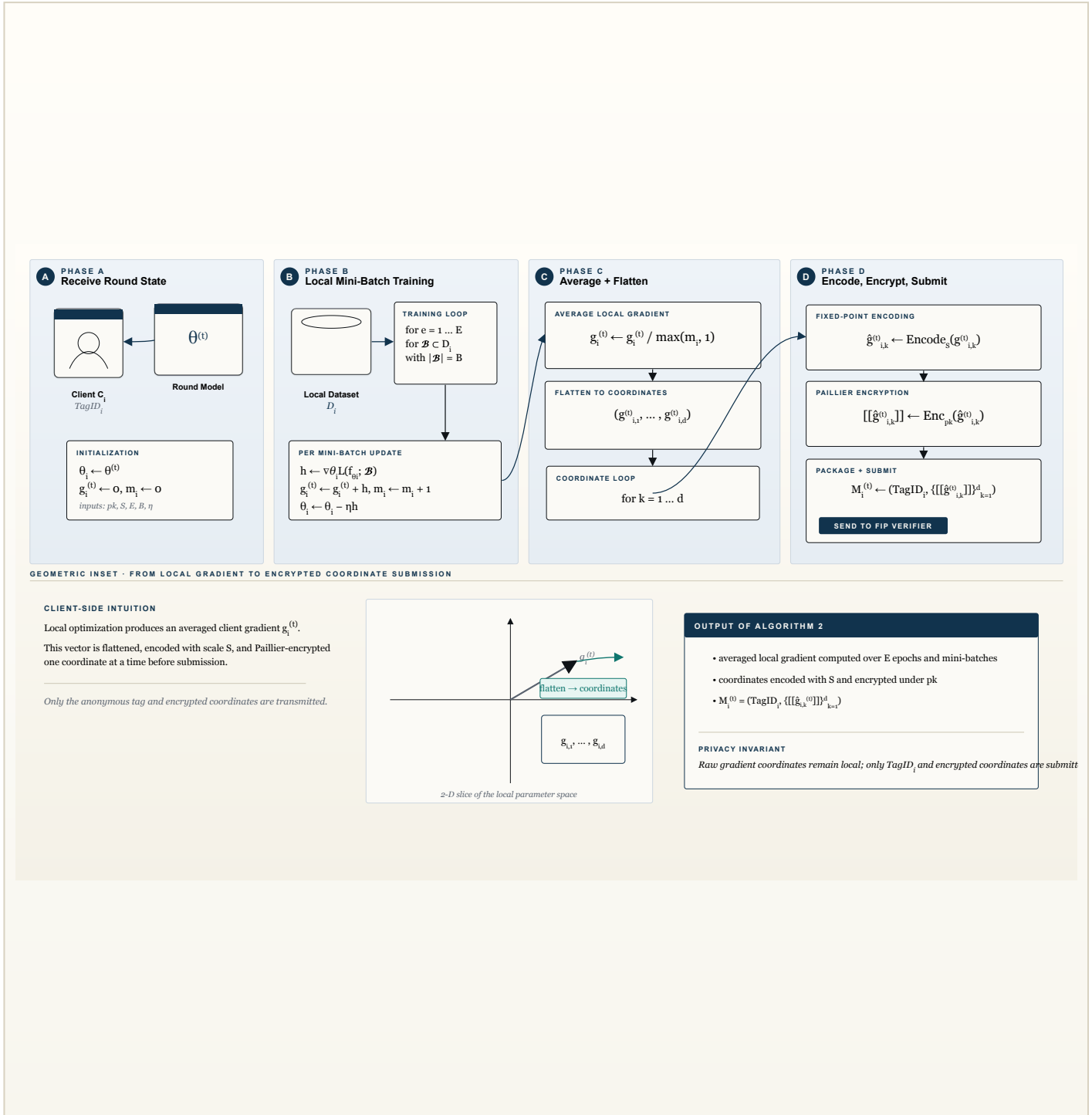


Figure 2. Algorithm 2. Client C_i receives the round model $\theta^{(t)}$, public key pk , fixed-point scale S , and local hyperparameters. It initializes $\theta_i \leftarrow \theta^{(t)}$, performs local mini-batch SGD over E epochs, accumulates and averages the local gradient $g_i^{(t)}$, and then flattens it into coordinates. Each coordinate is encoded as $\hat{g}_{i,k}^{(t)}$ using S and encrypted as $[[\hat{g}_{i,k}^{(t)}]]$ under pk . The transmitted message $M_i^{(t)} = (\text{TagID}_i, \{[[\hat{g}_{i,k}^{(t)}]]\}_{k=1}^d)$ contains only the anonymous tag and encrypted encoded coordinates. The inset summarizes the local descent direction and the flatten-encode-encrypt pipeline.

Algorithm 2 Client-Side Local Training and Encrypted Gradient Submission

Require: Client C_i with data $\mathcal{D}_i$, round model $\theta^{(t)}$, public key pk , scale s , epochs E , batch size B , learning rate η

Ensure: Encrypted submission $\mathcal{M}_i^{(t)}$

```

1:  $\theta_i \leftarrow \theta^{(t)}$ 
2:  $\mathbf{g}_i^{(t)} \leftarrow \mathbf{0}$ ,  $m_i \leftarrow 0$ 
3: for  $e \leftarrow 1$  to  $E$  do
4:   for all mini-batches  $\mathcal{B} \subset \mathcal{D}_i$  of size  $B$  do
5:      $\mathbf{h} \leftarrow \nabla_{\theta} \mathcal{L}(f_{\theta_i}; \mathcal{B})$ 
6:      $\mathbf{g}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} + \mathbf{h}$ ,  $m_i \leftarrow m_i + 1$ 
7:      $\theta_i \leftarrow \theta_i - \eta \mathbf{h}$ 
8:   end for
9: end for
10:  $\mathbf{g}_i^{(t)} \leftarrow \mathbf{g}_i^{(t)} / \max(m_i, 1)$ 
11: Flatten  $\mathbf{g}_i^{(t)}$  into coordinates  $(g_{i,1}^{(t)}, \dots, g_{i,d}^{(t)})$ 
12: for  $k \leftarrow 1$  to  $d$  do
13:    $\hat{g}_{i,k}^{(t)} \leftarrow \text{ENCODE}_s(g_{i,k}^{(t)})$ 
14:    $\llbracket g_{i,k}^{(t)} \rrbracket \leftarrow \text{ENC}_{pk}(\hat{g}_{i,k}^{(t)})$ 
15: end for
16:  $\mathcal{M}_i^{(t)} \leftarrow (\text{TagID}_i, \{\llbracket g_{i,k}^{(t)} \rrbracket\}_{k=1}^d)$ 
17: Send  $\mathcal{M}_i^{(t)}$  to the FIP Verifier

```

5.3.2 Phase II: Homomorphic FIP Score Computation

The Verifier forms the projection onto $\mathbf{R}^{(t)}$ while the update is still encrypted. After Paillier scalar multiplication and accumulation, the TKA decrypts only $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{R}^{(t)} \rangle$; no coordinate of $\mathbf{g}_i^{(t)}$ is opened. Algorithm 3 gives the computation.

Homomorphic Functional Inner Product Score Computation

A geometric view of server-side encrypted scoring — the FIP Verifier combines encrypted encoded client coordinates with the encoded trusted reference, decrypts only the scalar accumulator through the TKA, decodes the signed fixed-point scalar score, and returns the round score map $\mathcal{S}^{(t)}$.

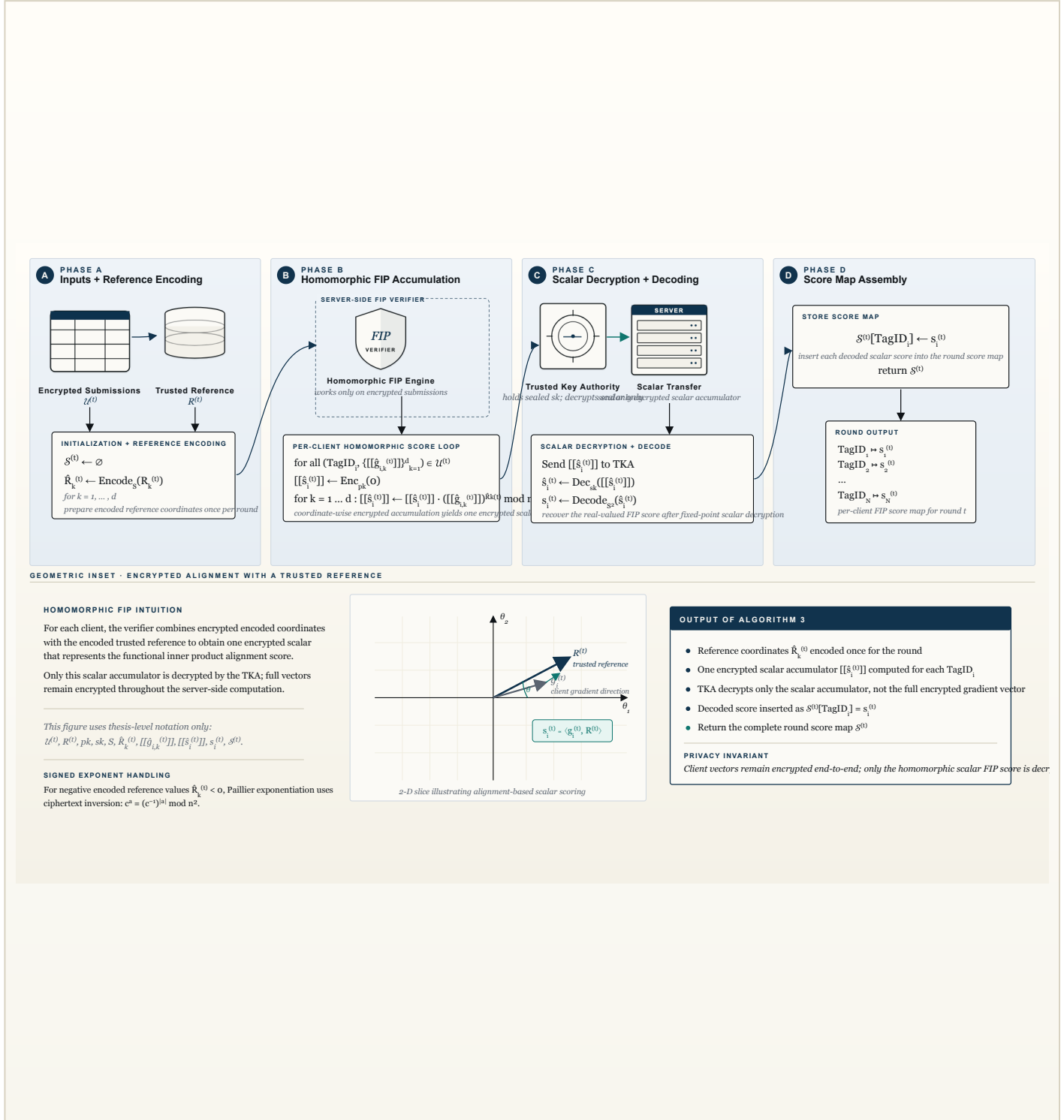


Figure 3. Algorithm 3. The FIP Verifier receives the encrypted submission set $\mathcal{U}^{(t)}$ and trusted reference $R^{(t)}$. It initializes the score map $\mathcal{S}^{(t)} = \emptyset$, encodes the reference coordinates as $\hat{R}_k^{(t)}$, and for each client accumulates an encrypted scalar inner-product score $[[\hat{s}_i^{(t)}]]$ using Paillier homomorphism. Only this scalar accumulator is sent to the TKA for decryption; the TKA returns $\hat{s}_i^{(t)}$, which is decoded as $s_i^{(t)}$ using $\text{Decode}_{\mathcal{S}}(\cdot)$. The verifier stores the result as $\mathcal{S}^{(t)}[\text{TagID}_i] \leftarrow s_i^{(t)}$. The inset illustrates alignment-based scalar scoring with respect to the

Algorithm 3 Homomorphic Functional Inner Product Score Computation

Require: Encrypted submissions $\mathcal{U}^{(t)}$, reference $\mathbf{R}^{(t)}$, public key pk , secret key sk held by TKA, scale s

Ensure: FIP score map $\mathcal{S}^{(t)}$

```

1:  $\mathcal{S}^{(t)} \leftarrow \emptyset$ 
2: Encode reference coordinates  $\widehat{R}_k^{(t)} \leftarrow \text{ENCODE}_s(R_k^{(t)})$  for  $k = 1, \dots, d$ 
3: for all  $(\text{TagID}_i, \{\llbracket g_{i,k}^{(t)} \rrbracket\}_{k=1}^d) \in \mathcal{U}^{(t)}$  do
4:    $\llbracket \widehat{s}_i^{(t)} \rrbracket \leftarrow \text{ENC}_{pk}(0)$ 
5:   for  $k \leftarrow 1$  to  $d$  do
6:      $\llbracket \widehat{s}_i^{(t)} \rrbracket \leftarrow \llbracket \widehat{s}_i^{(t)} \rrbracket \cdot (\llbracket g_{i,k}^{(t)} \rrbracket)^{\widehat{R}_k^{(t)}} \bmod n^2$ 
7:   end for
8:   Send  $\llbracket \widehat{s}_i^{(t)} \rrbracket$  to TKA for scalar decryption
9:    $\widehat{s}_i^{(t)} \leftarrow \text{DEC}_{sk}(\llbracket \widehat{s}_i^{(t)} \rrbracket)$ 
10:   $s_i^{(t)} \leftarrow \text{DECODE}_{s^2}(\widehat{s}_i^{(t)})$ 
11:   $\mathcal{S}^{(t)}[\text{TagID}_i] \leftarrow s_i^{(t)}$ 
12: end for
13: return  $\mathcal{S}^{(t)}$ 

```

5.3.3 Phase III: Two-Stage Geometric Audit

The audit is intentionally only two stages. First, the *direction filter* rejects non-positive scores. Second, among the positive scores, the *magnitude window* keeps only $\tau_{\min}^{(t)} \leq s_i^{(t)} \leq \tau_{\max}^{(t)}$. The lower floor catches weak positive updates; the upper cap catches large aligned updates. Algorithm 4 gives the procedure.

Two-Stage Geometric Audit for Malicious Client Detection

A geometric view of the audit logic — Stage 1 rejects directionally weak or negatively aligned clients using ϵ_{dir} , while Stage 2 applies robust median/MAD magnitude screening to the provisional pass set and returns accepted encrypted submissions $\mathcal{A}^{(t)}$ and rejected tags $\mathcal{D}^{(t)}$.

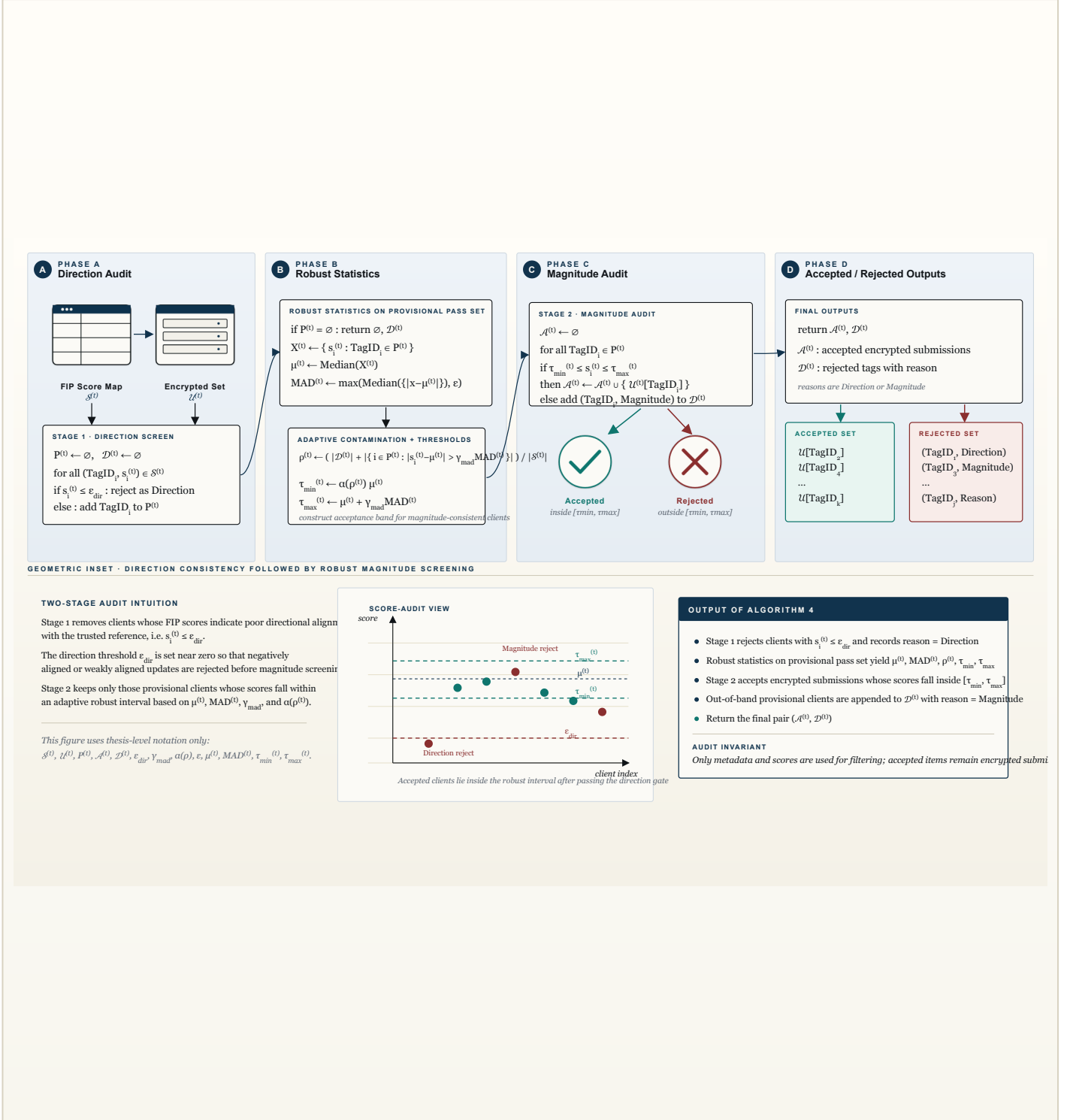


Figure 4. Algorithm 4. The verifier takes the FIP score map $\mathcal{S}^{(t)}$ and encrypted submission set $\mathcal{U}^{(t)}$. In Stage 1, it rejects clients with $s_i^{(t)} \leq \epsilon_{\text{dir}}$ as directionally suspicious and forms the provisional pass set $P^{(t)}$. In Stage 2, it computes the robust statistics $\mu^{(t)}$ and $\text{MAD}^{(t)}$, estimates the contamination rate $\rho^{(t)}$, and constructs the adaptive interval $[\tau_{\min}^{(t)}, \tau_{\max}^{(t)}]$. Encrypted submissions whose scores fall inside this band are retained in the accepted set $\mathcal{A}^{(t)}$, while the others are added to the rejected tag set $\mathcal{D}^{(t)}$ with reason Direction or Magnitude. The inset visualizes this two-stage screening in score space.

Algorithm 4 Two-Stage Geometric Audit for Malicious Client Detection**Require:** FIP scores $\mathcal{S}^{(t)}$, encrypted submissions $\mathcal{U}^{(t)}$, direction threshold ϵ_{dir} ,MAD multiplier γ_{mad} , floor map $\alpha(\rho)$, stability floor ε **Ensure:** Accepted encrypted set $\mathcal{A}^{(t)}$, rejected tag set $\mathcal{D}^{(t)}$

```

1:  $\mathcal{P}^{(t)} \leftarrow \emptyset, \mathcal{D}^{(t)} \leftarrow \emptyset$ 
2: for all  $(\text{TagID}_i, s_i^{(t)}) \in \mathcal{S}^{(t)}$  do
3:   if  $s_i^{(t)} \leq \epsilon_{\text{dir}}$  then
4:      $\mathcal{D}^{(t)} \leftarrow \mathcal{D}^{(t)} \cup \{(\text{TagID}_i, \text{DIRECTION})\}$ 
5:   else
6:      $\mathcal{P}^{(t)} \leftarrow \mathcal{P}^{(t)} \cup \{\text{TagID}_i\}$ 
7:   end if
8: end for
9: if  $\mathcal{P}^{(t)} = \emptyset$  then
10:   return  $\emptyset, \mathcal{D}^{(t)}$ 
11: end if
12:  $\mathcal{X}^{(t)} \leftarrow \{s_i^{(t)} : \text{TagID}_i \in \mathcal{P}^{(t)}\}$ 
13:  $\mu^{(t)} \leftarrow \text{MEDIAN}(\mathcal{X}^{(t)})$ 
14:  $\text{MAD}^{(t)} \leftarrow \max(\text{MEDIAN}(\{|x - \mu^{(t)}| : x \in \mathcal{X}^{(t)}\}), \varepsilon)$ 
15:  $\rho^{(t)} \leftarrow \frac{|\mathcal{D}^{(t)}| + |\{i \in \mathcal{P}^{(t)} : |s_i^{(t)} - \mu^{(t)}| > \gamma_{\text{mad}} \text{MAD}^{(t)}\}|}{|\mathcal{S}^{(t)}|}$ 
16:  $\tau_{\min}^{(t)} \leftarrow \alpha(\rho^{(t)}) \mu^{(t)}$ 
17:  $\tau_{\max}^{(t)} \leftarrow \mu^{(t)} + \gamma_{\text{mad}} \text{MAD}^{(t)}$ 
18:  $\mathcal{A}^{(t)} \leftarrow \emptyset$ 
19: for all  $\text{TagID}_i \in \mathcal{P}^{(t)}$  do
20:   if  $\tau_{\min}^{(t)} \leq s_i^{(t)} \leq \tau_{\max}^{(t)}$  then
21:      $\mathcal{A}^{(t)} \leftarrow \mathcal{A}^{(t)} \cup \{\mathcal{U}^{(t)}[\text{TagID}_i]\}$ 
22:   else
23:      $\mathcal{D}^{(t)} \leftarrow \mathcal{D}^{(t)} \cup \{(\text{TagID}_i, \text{MAGNITUDE})\}$ 
24:   end if
25: end for
26: return  $\mathcal{A}^{(t)}, \mathcal{D}^{(t)}$ 

```

5.3.4 Phase IV: Secure Aggregation

The Aggregation Server adds only accepted ciphertext vectors. The TKA decrypts the aggregate vector, not the individual accepted gradients; rejected

submissions never need to be opened.

5.3.5 Phase V: Reference Update and Model Distribution

The final phase averages the accepted aggregate, updates the model, and refreshes the reference with momentum β . I normalise the reference before the next round so later scores are comparable. If no client is accepted, both the model and the reference are kept unchanged. Algorithm 5 combines these steps.

Secure Aggregation and Momentum-Based Reference Update

A geometric view of the final round update — accepted encrypted coordinates are homomorphically aggregated, only aggregate coordinates are decrypted by the TKA, the averaged global gradient updates the model, and a momentum-normalized trusted reference $R^{(t+1)}$ is stored for the next round.

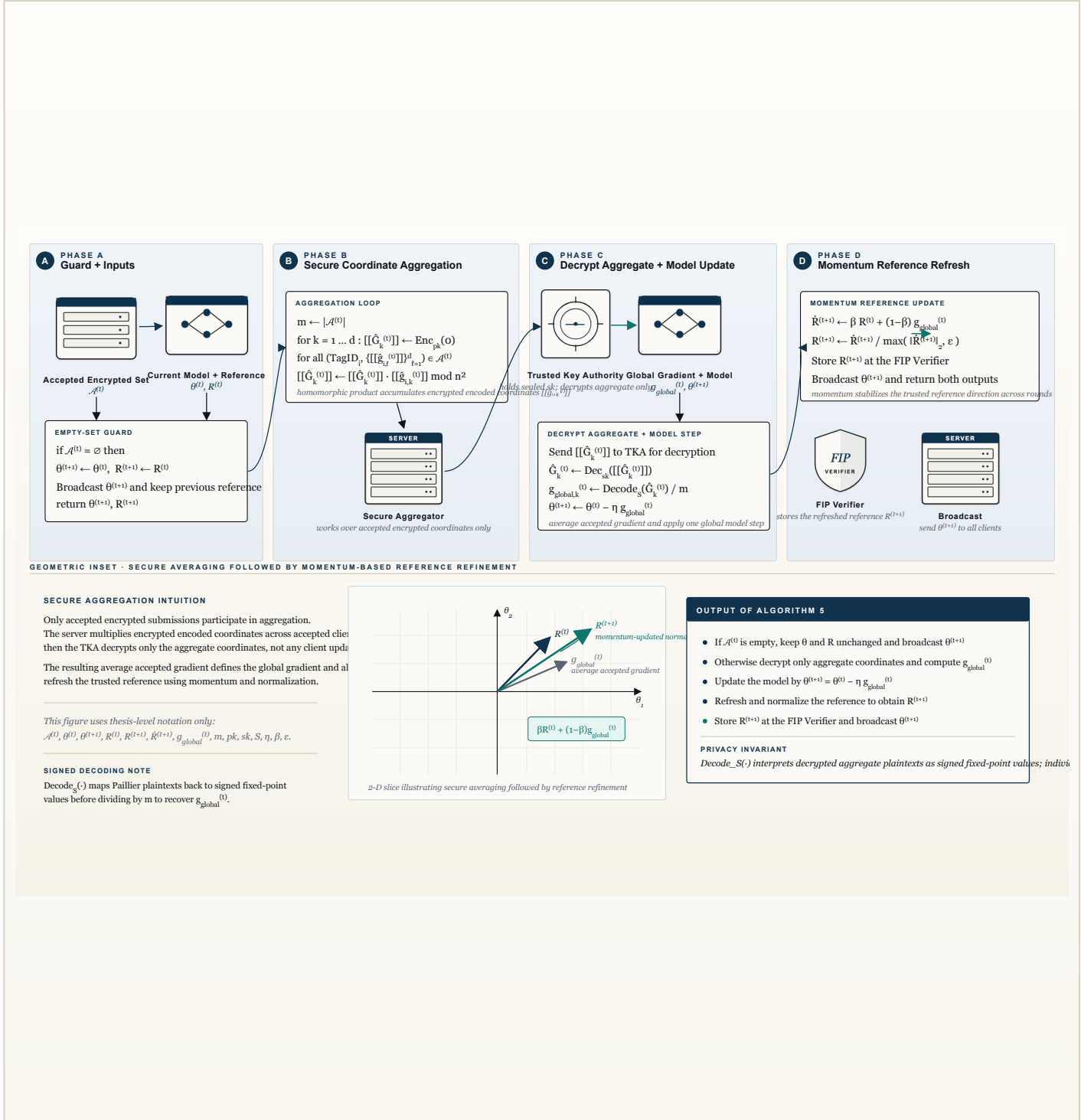


Figure 5. Algorithm 5. If the accepted encrypted set $\mathcal{A}^{(t)}$ is empty, the server keeps $\theta^{(t+1)} \leftarrow \theta^{(t)}$ and $R^{(t+1)} \leftarrow R^{(t)}$, broadcasts the unchanged model, and returns. Otherwise, for each coordinate k , the server homomorphically aggregates the accepted encrypted encoded values into $[[\hat{G}_k^{(t)}]]$ and sends only these aggregate ciphertexts to the TKA. After decryption and signed fixed-point decoding, the average accepted gradient $g_{\text{global}}^{(t)}$ is recovered and used to update the model by $\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta g_{\text{global}}^{(t)}$. The trusted reference is then refreshed as $\hat{R}^{(t+1)} \leftarrow \beta R^{(t)} + (1-\beta)g_{\text{global}}^{(t)}$ and normalized to $R^{(t+1)}$ using $\max(\|\hat{R}^{(t+1)}\|_2, \epsilon)$. The inset illustrates secure averaging followed by momentum-based reference refinement.

Algorithm 5 Secure Aggregation and Momentum-Based Reference Update

Require: Accepted encrypted set $\mathcal{A}^{(t)}$, model $\theta^{(t)}$, reference $\mathbf{R}^{(t)}$, public key pk , secret key sk held by TKA, scale s , learning rate η , momentum β , stability floor ε

Ensure: Updated model $\theta^{(t+1)}$, updated reference $\mathbf{R}^{(t+1)}$

```

1: if  $\mathcal{A}^{(t)} = \emptyset$  then
2:    $\theta^{(t+1)} \leftarrow \theta^{(t)}, \mathbf{R}^{(t+1)} \leftarrow \mathbf{R}^{(t)}$ 
3:   Broadcast  $\theta^{(t+1)}$  and keep the previous reference
4:   return  $\theta^{(t+1)}, \mathbf{R}^{(t+1)}$ 
5: end if
6:  $m \leftarrow |\mathcal{A}^{(t)}|$ 
7: for  $k \leftarrow 1$  to  $d$  do
8:    $\llbracket \hat{G}_k^{(t)} \rrbracket \leftarrow \text{ENC}_{pk}(0)$ 
9:   for all  $(\text{TagID}_i, \{g_{i,\ell}^{(t)}\}_{\ell=1}^d) \in \mathcal{A}^{(t)}$  do
10:     $\llbracket \hat{G}_k^{(t)} \rrbracket \leftarrow \llbracket \hat{G}_k^{(t)} \rrbracket \cdot \llbracket g_{i,k}^{(t)} \rrbracket \bmod n^2$ 
11:   end for
12:   Send  $\llbracket \hat{G}_k^{(t)} \rrbracket$  to TKA for aggregate decryption
13:    $\hat{G}_k^{(t)} \leftarrow \text{DEC}_{sk}(\llbracket \hat{G}_k^{(t)} \rrbracket)$ 
14:    $g_{\text{global},k}^{(t)} \leftarrow \text{DECODE}_s(\hat{G}_k^{(t)})/m$ 
15: end for
16:  $\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \mathbf{g}_{\text{global}}^{(t)}$ 
17:  $\tilde{\mathbf{R}}^{(t+1)} \leftarrow \beta \mathbf{R}^{(t)} + (1 - \beta) \mathbf{g}_{\text{global}}^{(t)}$ 
18:  $\mathbf{R}^{(t+1)} \leftarrow \tilde{\mathbf{R}}^{(t+1)} / \max(\|\tilde{\mathbf{R}}^{(t+1)}\|_2, \varepsilon)$ 
19: Store  $\mathbf{R}^{(t+1)}$  at the FIP Verifier and broadcast  $\theta^{(t+1)}$ 
20: return  $\theta^{(t+1)}, \mathbf{R}^{(t+1)}$ 

```

5.4 Theoretical Analysis

The three guarantees below formalise the privacy, admissibility, and descent conditions introduced in Section 4.6.

5.4.1 Gradient Privacy

Theorem 5.1 (Gradient Privacy). *In round t , the FIP Verifier reveals one scalar for client i , $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{R}^{(t)} \rangle \in \mathbb{R}$. When $d \gg 1$, this scalar does not determine $\mathbf{g}_i^{(t)}$: the equation $\mathbf{R}^{(t)\top} \mathbf{g}_i^{(t)} = s_i^{(t)}$ leaves a $(d - 1)$ -dimensional affine set of possible gradients. Equivalently, $I(\mathbf{g}_i^{(t)}; s_i^{(t)}) \leq H(s_i^{(t)}) \ll H(\mathbf{g}_i^{(t)})$.*

Proof. The Verifier applies only the homomorphic inner product in (4.8), producing $\llbracket s_i^{(t)} \rrbracket$. Once $s_i^{(t)}$ is known, every vector in the affine set $\{\mathbf{g} : \langle \mathbf{g}, \mathbf{R}^{(t)} \rangle = s_i^{(t)}\}$ is consistent with the observation. Any component orthogonal to $\mathbf{R}^{(t)}$ is hidden by the scalar score, and the stated mutual-information bound follows by data processing. $\square$

5.4.2 Attack Resilience

Theorem 5.2 (Attack Resilience). *Any malicious update $\mathbf{g}_{adv}$ accepted by the FIP-FL filter in round t must satisfy simultaneously*

$$\langle \mathbf{g}_{adv}, \mathbf{R}^{(t)} \rangle > 0 \quad (\text{direction constraint}), \quad (5.1)$$

$$\tau_{\min}^{(t)} \leq \langle \mathbf{g}_{adv}, \mathbf{R}^{(t)} \rangle \leq \tau_{\max}^{(t)} \quad (\text{magnitude constraint}), \quad (5.2)$$

where $\tau_{\min}^{(t)}$ and $\tau_{\max}^{(t)}$ are computed from the round-wise positive-score distribution. The two constraints jointly rule out direction-reversal, scaling-to-zero, and destabilising large-norm attacks while keeping the audit independent of any external auditor.

Proof. The geometric audit first applies the non-positive direction check and then the $\tau_{\min}^{(t)}/\tau_{\max}^{(t)}$ magnitude window. An adversary failing either clause is rejected. A direct poisoning gradient $\mathbf{g}_{adv} \approx -\mathbf{g}_{\text{honest}}$ gives a non-positive score, a near-zero scaling attack falls below $\tau_{\min}^{(t)}$, and a destabilising large-norm attack exceeds $\tau_{\max}^{(t)} = \mu^{(t)} + 5 \cdot \text{MAD}^{(t)}$. $\square$

5.4.3 Convergence Guarantee

Theorem 5.3 (Descent under L -Smoothness). *Let $F(\mathbf{w})$ be L -smooth and let $\Delta_{\text{global}}^{(t)}$ be the mean of the accepted updates in round t . If the accepted aggregate*

is descent-aligned with the true gradient,

$$\langle \nabla F(\mathbf{w}^{(t)}), \Delta_{\text{global}}^{(t)} \rangle > 0, \quad (5.3)$$

then any learning rate

$$0 < \eta < \frac{2\langle \nabla F(\mathbf{w}^{(t)}), \Delta_{\text{global}}^{(t)} \rangle}{L\|\Delta_{\text{global}}^{(t)}\|_2^2} \quad (5.4)$$

gives a strict one-round loss decrease:

$$F(\mathbf{w}^{(t+1)}) < F(\mathbf{w}^{(t)}). \quad (5.5)$$

Consequently, if this alignment holds in expectation over accepted rounds, the expected global loss decreases monotonically.

Proof. The L -smoothness descent lemma gives

$$F(\mathbf{w}^{(t)} - \eta \Delta_{\text{global}}^{(t)}) \leq F(\mathbf{w}^{(t)}) - \eta \langle \nabla F(\mathbf{w}^{(t)}), \Delta_{\text{global}}^{(t)} \rangle + \frac{L\eta^2}{2} \|\Delta_{\text{global}}^{(t)}\|_2^2. \quad (5.6)$$

Under the stated alignment condition, the right-hand side is strictly below $F(\mathbf{w}^{(t)})$ whenever η satisfies the stated bound. The FIP-FL filter supports this condition by admitting only updates with positive reference projection and bounded scalar magnitude; the theorem states the resulting descent requirement explicitly rather than assuming the reference is perfectly equal to the true gradient. $\square$

These results give the criteria I check experimentally: the full gradient should stay hidden, harmful or over-scaled projections should be filtered, and the accepted aggregate should keep the descent orientation required by the smoothness argument.

6 Experimental Setup

This chapter records the exact protocol I used for the FIP-FL runs: datasets, client splits, attacks, metrics, software, and hardware.

6.1 Datasets

I evaluate the method on one image benchmark and one intrusion-detection benchmark, summarized in Table 6.1.

Table 6.1: Datasets used in the evaluation of FIP-FL.

Dataset	Modality	Classes	Features	Samples
MNIST	Image	10	784	60,000/10,000
KDDCup99	Tabular	5	41	$\approx$ 494K (10%)

MNIST. MNIST has 60,000 training and 10,000 test images. I flatten each 28×28 image into 784 features and scale pixels to $[0, 1]$. I keep MNIST in the study because Yazdinejad et al. [1] report the auditor-based result on the same benchmark.

KDDCup99. KDDCup99 is the tabular case. I use the standard 10% subset, about 494,000 records from the 4.9 million-record corpus. Each record has 41 traffic/protocol features, and I group the labels into NORMAL, DoS, PROBE, R2L, and U2R. Categorical fields are one-hot encoded, continuous fields are standardized, and the strong class imbalance gives a useful stress case for the magnitude window.

Model architecture. I use softmax logistic regression for both datasets, as in the SenSys 2026 poster and the auditor-based baseline. The parameter dimension is $d = (\text{input_dim}) \cdot C + C$, including bias terms. This keeps the Paillier workload manageable with a 1024-bit modulus and fits the convex L -smooth analysis in Theorem 5.3.

6.2 Data Partitioning

Each dataset is split into $N = 10$ disjoint client shards. For IID runs, I shuffle the training set and assign equal shards, so each client gets approximately $|\mathcal{D}_{\text{train}}|/N$ samples and the global label mix in expectation.

For non-IID runs, I use label skew. Client i receives a primary label set $\mathcal{L}_i$ of size $\lceil C/2 \rceil$, with $\lfloor C/4 \rfloor$ overlap between neighbouring clients. I draw 80% of that client's data from $\mathcal{L}_i$ and 20% from all classes. This follows the SenSys 2026 protocol and Yazdinejad et al. [1]. It is deliberately difficult because honest clients can point in different, still useful, descent directions.

6.3 Attack Models

I use two pre-training label-flipping attacks. Both change labels before gradient computation and do not need the Paillier key or the FIP score.

Untargeted label flipping. A malicious client relabels each local sample by a fixed cyclic offset:

$$y_k^{(i, \text{mal})} = (y_k^{(i)} + \kappa) \bmod C. \quad (6.1)$$

For MNIST, $\kappa = 5$ gives a digit permutation with no fixed point. For KDDCup99, $\kappa = 2$ matches the five-class collapsed task. In both cases, malicious clients train toward a deliberately wrong objective.

Targeted label flipping. A malicious client changes only a source class c_s to a target class c_t :

$$y_k^{(i, \text{mal})} = \begin{cases} c_t & \text{if } y_k^{(i)} = c_s, \\ y_k^{(i)} & \text{otherwise.} \end{cases} \quad (6.2)$$

For MNIST I use $(c_s, c_t) = (0, 7)$. For KDDCup99 I choose the analogous source–target pair from the dominant classes in the skewed split. Because this attack damages one class while leaving the rest of the task plausible, target-class accuracy is the main diagnostic.

Malicious-client rates. I sweep $f/N \in \{10\%, 20\%, 30\%, 50\%\}$ for each dataset, partition, and attack. The 50% row is the headline stress case because it is close to the limit where majority-based filtering becomes fragile.

6.4 Evaluation Metrics

I report five metrics: three for held-out utility and two for verifier-side detection.

Target accuracy (T_{acc}). Accuracy restricted to test samples from the targeted source class c_s :

$$T_{\text{acc}} = \frac{|\{(x, y) \in \mathcal{D}_{\text{test}} : y = c_s, \hat{f}(x) = c_s\}|}{|\{(x, y) \in \mathcal{D}_{\text{test}} : y = c_s\}|}. \quad (6.3)$$

A targeted attack tries to drive this value down. For untargeted attacks, I use class 0 as a fixed diagnostic class.

Other-label accuracy (O_{acc}). Accuracy on all test samples outside the targeted source class:

$$O_{\text{acc}} = \frac{|\{(x, y) \in \mathcal{D}_{\text{test}} : y \neq c_s, \hat{f}(x) = y\}|}{|\{(x, y) \in \mathcal{D}_{\text{test}} : y \neq c_s\}|}. \quad (6.4)$$

This separates source-class damage from performance on the rest of the task.

Overall accuracy (ACC). Standard test accuracy over all classes:

$$\text{ACC} = \frac{|\{(x, y) \in \mathcal{D}_{\text{test}} : \hat{f}(x) = y\}|}{|\mathcal{D}_{\text{test}}|}. \quad (6.5)$$

This is the primary utility metric for untargeted attacks.

True positive rate (TPR). Fraction of malicious clients rejected by the FIP-FL audit:

$$\text{TPR} = \frac{|\{i : \text{client } i \text{ is malicious and is rejected}\}|}{|\{i : \text{client } i \text{ is malicious}\}|}. \quad (6.6)$$

TPR measures malicious-client rejection; $\text{TPR} = 1.00$ means the audit filtered

every malicious client in that run.

False positive rate (FPR). Fraction of honest clients rejected by the FIP-FL audit:

$$\text{FPR} = \frac{|\{i : \text{client } i \text{ is honest and is rejected}\}|}{|\{i : \text{client } i \text{ is honest}\}|}. \quad (6.7)$$

FPR measures the honest-client cost of filtering. I average metrics over three independent seeds and report a standard deviation when it exceeds 0.5 percentage points.

6.5 Implementation and Hardware

The implementation is in Python 3 with NumPy, scikit-learn, and phe using a 1024-bit Paillier modulus. I run the FIP Verifier and Aggregation Server as separate Python processes connected by in-memory queues; clients are worker processes. Encryption and FIP-score computation use multiprocessing.

Unless stated otherwise, I use $R = 300$, $B = 50$, $E = 1$, $\eta = 0.01$, fixed-point scale $s = 10,000$, direction threshold $\epsilon_{\text{dir}} = 0$, MAD multiplier $\gamma_{\text{mad}} = 5$, stability floor $\varepsilon = 10^{-6}$, and reference momentum $\beta = 0.80$. The adaptive lower-floor ratio $\alpha(\rho)$ takes 0.55, 0.50, 0.40, and 0.30 across the four increasing attack-rate bands.

Wall-clock measurements in Chapter 8 use a workstation with a 25-core CPU, 128 GB RAM, and a Linux-based operating system. The $13\times$ speed-up cited in the abstract is measured on the Phase II homomorphic inner-product step against the naive single-core implementation.

7 Results and Analysis

This chapter reports the completed FIP-FL grid from Chapter 6. I use the round-300 values for overall accuracy (ACC), target-class accuracy (T_{acc}), other-label accuracy (O_{acc}), true positive rate (TPR), and false positive rate (FPR).

7.1 Non-IID Partition, Untargeted Attacks

This is the hardest untargeted setting in my grid: honest gradients already spread because of label skew, and malicious clients add gradients trained on cyclically flipped labels. I read overall accuracy as utility and TPR/FPR as filtering behaviour.

7.1.1 MNIST

Table 7.1: MNIST, non-IID partition, untargeted label-flipping attack.

AR	ACC (%)	T_{acc} (%)	O_{acc} (%)	TPR	FPR
10%	91.22	97.60	90.18	1.00	0.00
20%	91.02	97.87	89.94	1.00	0.00
30%	90.78	97.33	89.31	1.00	0.00
50%	91.10	97.51	89.91	1.00	0.20

7.1.2 KDDCup99

Table 7.2: KDDCup99, non-IID partition (Dirichlet $\alpha = 0.5$), untargeted label-flipping attack.

AR	ACC (%)	T_{acc} (%)	O_{acc} (%)	TPR	FPR
10%	99.71	99.50	84.62	1.00	0.00
20%	99.72	99.55	83.65	1.00	0.00
30%	99.69	99.45	85.58	1.00	0.14
50%	99.69	99.70	77.88	1.00	0.40

Observations. In both non-IID untargeted tables, TPR is 1.00 for every attack rate. MNIST stays within 90.78–91.22% ACC, and false rejection appears only at the 50% row. KDDCup99 remains above 99.6% ACC; its higher FPR at the heaviest load shows where honest label-skewed gradients come closest to weak positive malicious scores.

7.2 Non-IID Partition, Targeted Attacks

In targeted label flipping, the main question is whether the source class survives. I therefore treat T_{acc} as the primary diagnostic.

7.2.1 MNIST

Table 7.3: MNIST, non-IID, targeted label-flipping attack ($c_s = 0, c_t = 7$); SenSys 2026 headline targeted-accuracy cell.

AR	Undefended (%)	Auditor-based (%)	FIP-FL (%)
50%	≈ 7.00	96.50	97.69

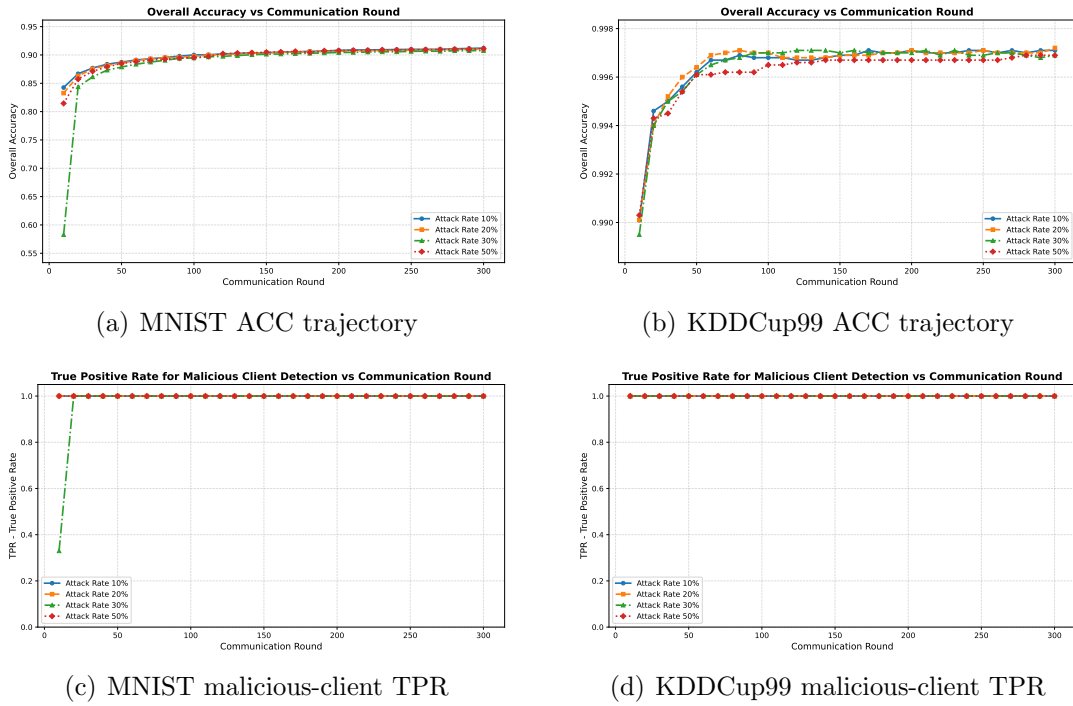


Figure 7.1: Non-IID untargeted attack dynamics over 300 communication rounds. Accuracy converges stably for both datasets, while the direction/magnitude audit reaches perfect malicious-client detection by the final round.

7.2.2 KDDCup99

Table 7.4: KDDCup99, non-IID partition (2 shards/client), targeted label-flipping attack.

AR	ACC (%)	T_{acc} (%)	O_{acc} (%)	TPR	FPR
10%	99.54	99.75	63.46	1.00	0.00
20%	99.69	99.65	78.85	1.00	0.00
30%	99.70	99.50	82.69	1.00	0.00
50%	99.52	98.41	86.54	1.00	0.00

Observations. The MNIST row is the accepted-poster headline result. At 50% malicious clients, FIP-FL raises T_{acc} from roughly 7% without defence to 97.69%, above the 96.5% auditor-based value of Yazdinejad et al. [1]. On KDDCup99, the same sweep keeps TPR at 1.00, FPR at 0.00, and $T_{\text{acc}} = 98.41\%$ in the 50%

row.

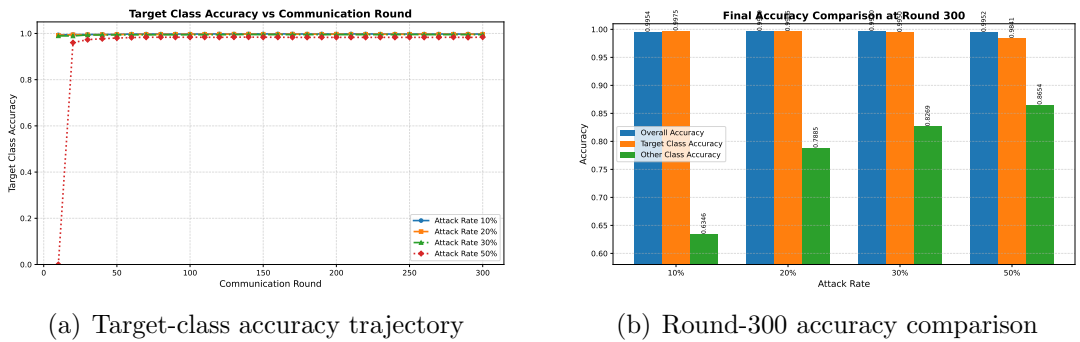


Figure 7.2: KDDCup99 non-IID targeted attack results. The target-class trajectory remains high through training, and the round-300 comparison shows that overall and target-class accuracy stay stable even as the malicious-client fraction increases.

7.3 IID Partition, Untargeted Attacks

IID partitions are my sanity-check regime: honest clients are statistically similar, so the FIP score distribution should be tighter than in label-skewed runs.

7.3.1 MNIST

Table 7.5: MNIST, IID partition, untargeted label-flipping attack.

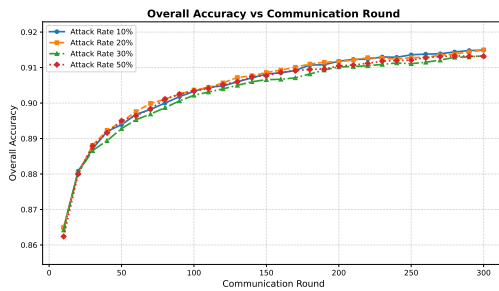
AR	ACC (%)	T_{acc} (%)	O_{acc} (%)	TPR	FPR
10%	91.49	97.51	90.40	1.00	0.00
20%	91.50	97.51	90.39	1.00	0.00
30%	91.33	97.42	90.28	1.00	0.00
50%	91.32	97.42	90.22	1.00	0.00

7.3.2 KDDCup99

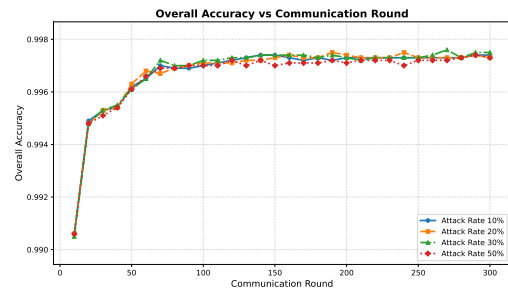
Table 7.6: KDDCup99, IID partition, untargeted label-flipping attack.

AR	ACC (%)	T_{acc} (%)	O_{acc} (%)	TPR	FPR
10%	99.74	99.60	85.58	1.00	0.00
20%	99.73	99.55	85.58	1.00	0.00
30%	99.75	99.60	85.58	1.00	0.00
50%	99.73	99.55	85.58	1.00	0.00

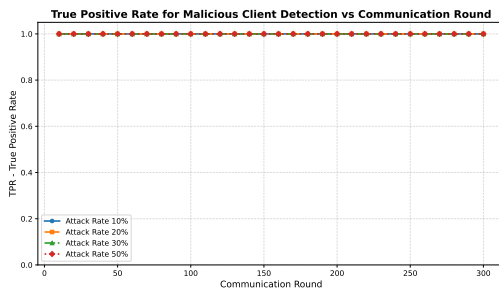
Observations. The IID untargeted regime behaves as expected. TPR is 1.00, FPR is 0.00, MNIST ACC varies by less than 0.2 percentage points, and KDDCup99 is almost unchanged across the sweep.



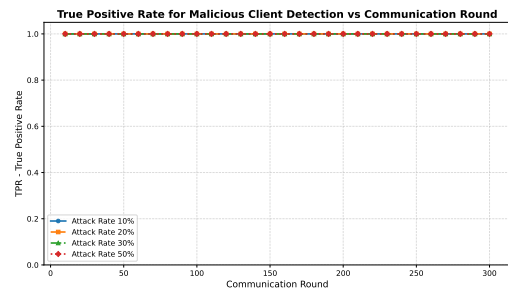
(a) MNIST ACC trajectory



(b) KDDCup99 ACC trajectory



(c) MNIST malicious-client TPR



(d) KDDCup99 malicious-client TPR

Figure 7.3: IID untargeted attack dynamics over 300 communication rounds. Both datasets show stable convergence, and the audit maintains perfect malicious-client detection without false positives at the reported final round.

7.4 IID Partition, Targeted Attacks

7.4.1 MNIST

Table 7.7: MNIST, IID partition, targeted label-flipping attack ($c_s = 0, c_t = 7$).

AR	ACC (%)	T_{acc} (%)	O_{acc} (%)	TPR	FPR
10%	91.62	97.51	90.57	1.00	0.00
20%	91.42	97.42	90.35	1.00	0.00
30%	91.32	97.42	90.22	1.00	0.00
50%	91.43	97.42	90.35	1.00	0.00

7.4.2 KDDCup99

Table 7.8: KDDCup99, IID partition, targeted label-flipping attack.

AR	ACC (%)	T_{acc} (%)	O_{acc} (%)	TPR	FPR
10%	99.73	99.60	84.62	1.00	0.00
20%	99.73	99.55	85.58	1.00	0.00
30%	99.72	99.50	85.58	1.00	0.00
50%	99.73	99.55	85.58	1.00	0.00

Observations. IID targeted runs match the IID untargeted pattern: exact detection, no final-round false rejection, MNIST $T_{acc} \geq 97.42\%$, and KDDCup99 ACC between 99.72% and 99.73%.

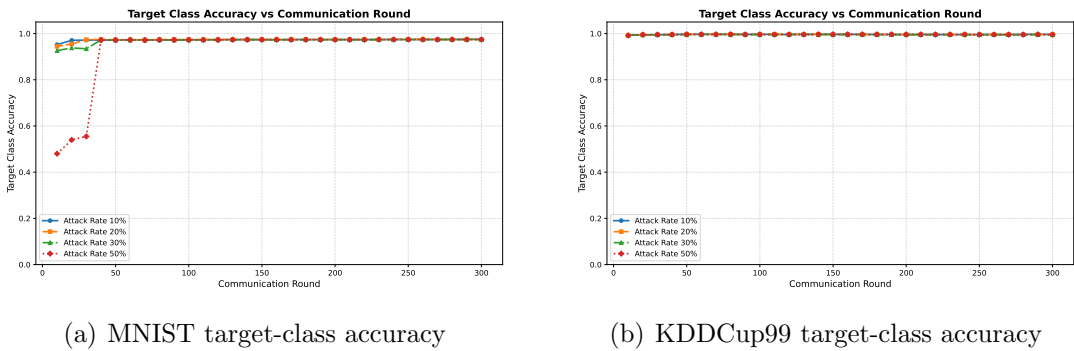


Figure 7.4: IID targeted attack target-class accuracy over 300 communication rounds. In both datasets, the target class recovers to a stable high-accuracy regime by the final round.

7.5 Aggregate Findings Across the Grid

The grid gives three takeaways. First, every reported row reaches $TPR = 1.00$, including the 50% malicious-client cases. Second, false rejection is confined to heavy non-IID untargeted load, where accuracy is still stable. Third, the same scalar FIP audit captures both targeted and untargeted attacks in the evaluated settings. Chapter 8 compares these results with the auditor-based baseline.

8 Comparative Analysis

This chapter compares FIP-FL with the defences reviewed in Chapter 3. I focus most closely on Yazdinejad et al. [1], because their system is the nearest encrypted-gradient baseline. The comparison follows five points: what gets decrypted, verifier cost, post-filter communication, non-IID behaviour, and MNIST non-IID targeted accuracy.

8.1 Privacy

The auditor-based scheme decrypts cluster-level statistics in each round, including the projected Gaussian-mixture representation and the distance statistics used for screening. Even if no individual gradient is reconstructed, those means and covariances describe the client population, which matters under non-IID data.

FIP-FL decrypts only one scalar per client per round, $s_i^{(t)} = \langle \mathbf{g}_i^{(t)}, \mathbf{R}^{(t)} \rangle$. It does not decrypt cluster means, covariance matrices, projection coefficients, or pairwise distances. The model update is opened only after accepted ciphertexts have been aggregated.

8.2 Computational Complexity

Table 8.1: Per-round computational cost. N : clients; $|\mathcal{A}|$: accepted clients; d : model dimension; B : batch size; E : local epochs.

Entity	Auditor-based [1]	FIP-FL
Client	$\mathcal{O}(dBE)$	$\mathcal{O}(dBE)$
Verifier / Auditor	$\mathcal{O}(NKD + Nd \log d) + \mathcal{O}(d^3)$	$\mathcal{O}(Nd + N \log N)$
Aggregation Server	$\mathcal{O}(dN \log N)$	$\mathcal{O}(\mathcal{A} d)$

Client cost is unchanged because both schemes run local SGD. The difference is at screening: the auditor-based method pays for projection, mixture fitting, and covariance inversion, while FIP-FL computes N homomorphic inner products and median/MAD statistics. After filtering, the server sums only accepted ciphertext vectors, giving $\mathcal{O}(|\mathcal{A}|d)$ work.

8.3 Communication Complexity

Table 8.2: Per-round communication cost by directed link.

Link	Auditor-based [1]	FIP-FL
Client $\rightarrow$ Verifier	$\mathcal{O}(Nd)$	$\mathcal{O}(Nd)$
Verifier $\rightarrow$ Server	$\mathcal{O}(L(Z + N))$	$\mathcal{O}(\mathcal{A} d)$
Server $\rightarrow$ Clients	$\mathcal{O}(d)$	$\mathcal{O}(d)$

The first payload is the same because both designs submit encrypted updates. After verification, FIP-FL forwards only accepted ciphertexts and avoids the extra auditor path.

8.4 Robustness and Engineering Trade-Offs

The auditor-based detector benefits when honest clients form a tighter and more populous cluster than malicious clients. Label skew weakens that condition. FIP-FL avoids cluster fitting and instead tests each update against the trusted reference and the adaptive scalar window.

The reference update is deliberately conservative: only updates passing both checks enter the aggregate, and the next reference uses exponential moving average rather than full replacement. Phase II FIP computations are independent across clients, which explains the $13\times$ wall-clock speed-up on the 25-core workstation of Section 6.5.

8.5 Scheme-versus-Scheme Accuracy Comparison

Table 8.3: Accuracy on MNIST non-IID under a 50% malicious-client rate. Entries are percentages; baseline values are from [1].

Scheme	Targeted (T_{acc})	Untargeted (ACC)
Krum [8]	71.4	78.3
Auror [11]	32.5	49.9
PEFL [13]	46.4	57.4
Sybils / FoolsGold [15]	89.7	89.5
FL-Trust [12]	37.8	43.4
ShieldFL [16]	90.1	89.4
Auditor-based [1]	96.5	96.3
FIP-FL (this thesis)	97.69	88.82

In this comparison, FIP-FL gives the best targeted accuracy while using weaker trust assumptions than the auditor-based baseline. Its untargeted result is close

to FoolsGold and ShieldFL but below the auditor-based row, so untargeted recovery remains the clearest empirical target for future work.

9 Conclusion and Future Work

9.1 Summary of the Thesis

This thesis asked whether Byzantine filtering can be placed inside PPFL without adding an auditor that decrypts cluster statistics. FIP-FL answers by replacing cluster inspection with one functional inner product between an encrypted update and a momentum-tracked reference direction. The only decrypted per-client value is a scalar score, which is checked for direction and then for magnitude.

The framework combines Paillier encryption, homomorphic score computation, secure aggregation, and momentum-based reference updating. The analysis clarifies what the scalar reveals, what an accepted adversary must satisfy, and when the accepted aggregate gives descent. In the completed grid, FIP-FL detects every reported malicious-client configuration, raises MNIST non-IID target accuracy from roughly 7% undefended to 97.69% at 50% malicious clients, removes the auditor's $\mathcal{O}(d^3)$ covariance step, and gives a 13× Phase II speed-up.

9.2 Limitations and Future Work

FIP-FL deliberately uses one reference direction per round. This is why verification remains linear, but it can be restrictive when a federation has several honest descent directions. A multi-reference cone $\{\mathbf{R}_k^{(t)}\}_{k=1}^K$ is a natural extension, provided it is protected against cone-overlap exploits.

The present evaluation uses logistic-regression models, dimensions in the low thousands, a fixed client set, and one attack mode per run. A deep-network version would likely need packed homomorphic encryption such as BFV or CKKS [5]. A broader study should also include client churn, changing data

distributions, adaptive adversaries, and mixed attacks.

Finally, the first reference is computed from a small public bootstrap set. Future work could derive this reference from robust aggregate statistics while still avoiding individual-gradient decryption. End-to-end deployment would then test the balance among encryption cost, communication cost, and model utility.

Bibliography

- [1] Abbas Yazdinejad, Ali Dehghantanha, Hadis Karimipour, Gautam Srivastava, and Reza M. Parizi. A robust privacy-preserving federated learning model against model poisoning attacks. *IEEE Transactions on Information Forensics and Security*, 19:6693–6708, 2024.
- [2] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 54, pages 1273–1282, 2017.
- [3] Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.
- [4] Pascal Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Advances in Cryptology – EUROCRYPT ’99*, pages 223–238. Springer, 1999.
- [5] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437. Springer, 2017.
- [6] Eugene Bagdasaryan, Andreas Veit, Yiqing Hua, Deborah Estrin, and Vitaly Shmatikov. How to backdoor federated learning. In *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 2938–2948, 2020.
- [7] Arjun Nitin Bhagoji, Supriyo Chakraborty, Prateek Mittal, and Seraphin Calo. Analyzing federated learning through an adversarial lens. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 634–643, 2019.
- [8] Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine-tolerant gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

- [9] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter Bartlett. Byzantine-robust distributed learning: Towards optimal statistical rates. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 5650–5659, 2018.
- [10] El Mahdi El Mhamdi, Rachid Guerraoui, and Sébastien Rouault. The hidden vulnerability of distributed learning in Byzantium. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 3521–3530, 2018.
- [11] Shiqi Shen, Shruti Tople, and Prateek Saxena. Auror: Defending against poisoning attacks in collaborative deep learning systems. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*, pages 508–519, 2016.
- [12] Xiaoyu Cao, Minghong Fang, Jia Liu, and Neil Zhenqiang Gong. FLTrust: Byzantine-robust federated learning via trust bootstrapping. In *Network and Distributed System Security Symposium (NDSS)*, 2021.
- [13] Xiaoyuan Liu, Hongwei Li, Guowen Xu, Zongqi Chen, Xiaoming Huang, and Rongxing Lu. Privacy-enhanced federated learning against poisoning adversaries. *IEEE Transactions on Information Forensics and Security*, 16: 4574–4588, 2021.
- [14] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*, pages 1175–1191, 2017.
- [15] Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. The limitations of federated learning in Sybil settings. In *23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, pages 301–316, 2020.
- [16] Zhuoran Ma, Jianfeng Ma, Yinbin Miao, Yuan Li, and Robert H. Deng. ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning. *IEEE Transactions on Information Forensics and Security*, 17:1639–1654, 2022.
- [17] Anthony Zhou and Amir Barati Farimani. Hamiltonian neural PDE solvers through functional approximation, 2025.